Smart Building Sensor Monitoring System

Author: Maral Torabi | Student ID: 40213312

A modular, enterprise-grade, object-oriented Python application designed for monitoring smart facility infrastructures, analyzing environmental sensor telemetry, evaluating multi-tiered threshold anomalies, tracking hardware lifecycles, and dispatching multi-channel facility alerts.

The codebase is engineered with strict separation of concerns, strong typing, clean domain abstractions, and distinct per-entity module files optimized for **UML reverse engineering** (e.g., Software Ideas Modeler) and **round-trip software engineering**.

1. Project Overview

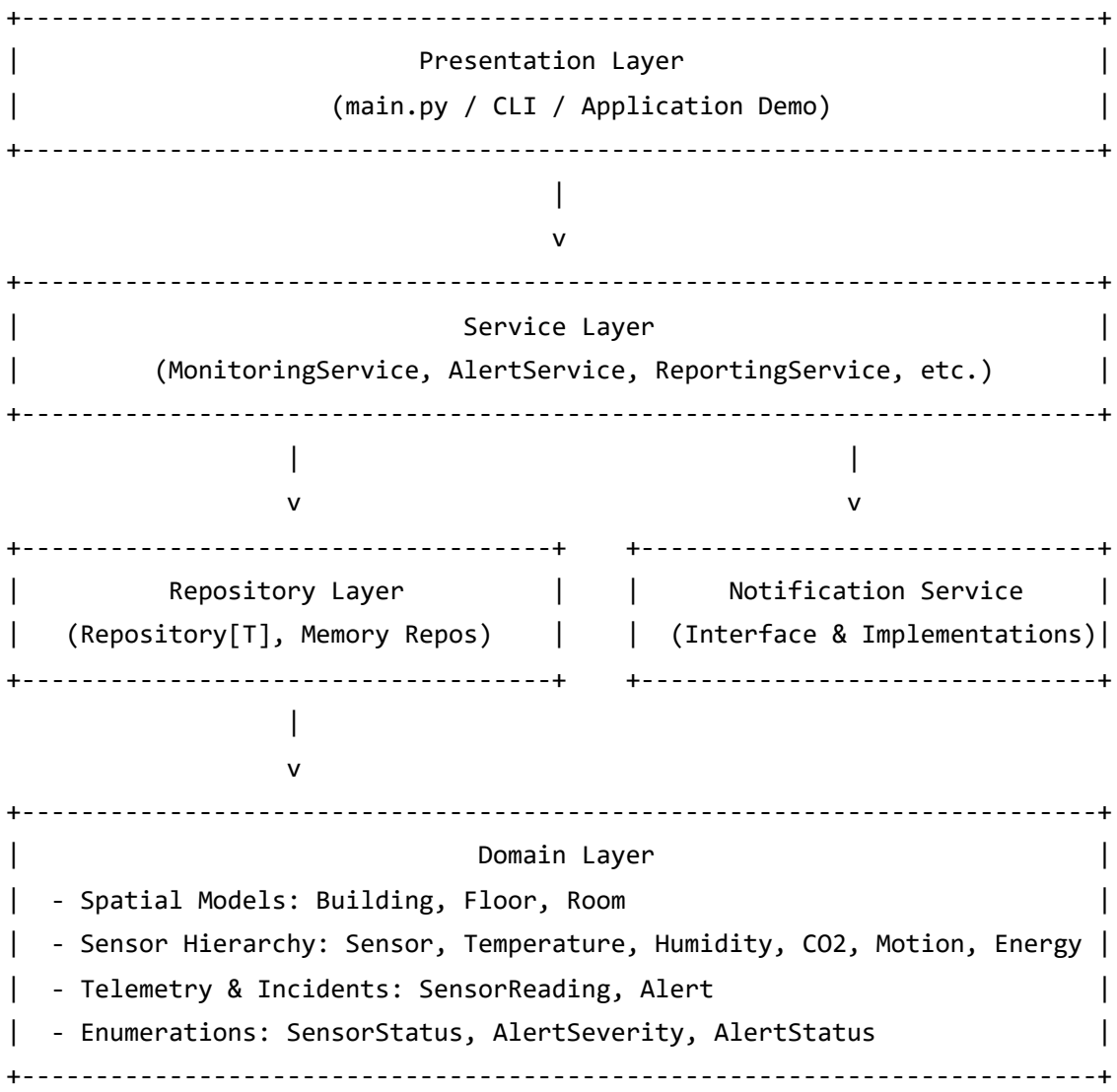
Modern smart buildings rely on hundreds of distributed sensor endpoints (monitoring temperature, humidity, carbon dioxide concentration, occupancy motion, and electrical power consumption) across multiple building floors and rooms.

The **Smart Building Sensor Monitoring System** provides:

- **Spatial Facility Modeling**: Full physical containment hierarchy representing Buildings, Floors, and Rooms.
- **Polymorphic Sensor Hierarchy**: Abstract and specialized sensor implementations for distinct telemetry metrics and automated reading generation.
- **Real-Time Stream Monitoring**: Continuous rule-based threshold evaluation against warning and critical boundaries.
- **Device Health & Offline Detection**: Automatic heartbeat tracking that flags unresponsive sensors as `OFFLINE` and triggers critical alarms.
- **Alert Lifecycle Management**: Comprehensive incident lifecycle tracking across `OPEN`, `ACKNOWLEDGED`, and `RESOLVED` states.
- **Pluggable Notifications**: Decoupled delivery mechanisms (Console, Email) adhering to the Dependency Inversion Principle.
- **Facility Analytics & Reporting**: Statistical metrics calculation (min, max, average) and executive health summary reports.

2. Layered Architecture

The system follows a strict **Layered Architecture** ensuring loose coupling, high cohesion, and testability:



1. **Domain Layer (`src/domain/`)**: Pure domain models, business logic, and entity definitions with zero external framework dependencies.
2. **Repository Layer (`src/repository/`)**: Data persistence abstraction utilizing the Generic Repository Pattern (`Repository[T]`), decoupling domain operations from data stores.
3. **Service Layer (`src/service/`)**: Orchestrates business workflows, stream processing, threshold analysis, device heartbeat auditing, and diagnostic reports.
4. **Notification Layer (`src/service/notification.py`)**: Abstract delivery contract enabling pluggable notification sinks (Console, SMTP Email, etc.).

3. Directory Structure

```
smart_building/
├── src/
│   ├── __init__.py
│   ├── domain/
│   │   ├── __init__.py          # Domain package exports (__all__)
│   │   ├── enums.py             # SensorStatus, AlertSeverity, AlertStatus
│   │   ├── sensor_reading.py    # SensorReading entity & unit conversions
│   │   ├── sensor.py            # Sensor (ABC), Temperature, Humidity, CO2, Motion, Energy
│   │   ├── alert.py             # Alert entity & lifecycle management
│   │   ├── room.py              # Room spatial model & sensor aggregation
│   │   ├── floor.py             # Floor spatial model & room containment
│   │   └── building.py          # Building spatial model & floor composite
│   ├── repository/
│   │   ├── __init__.py          # Repository package exports
│   │   ├── base.py              # Generic Repository[T] abstract interface
│   │   └── memory.py            # InMemoryRepository, SensorRepo, ReadingRepo, AlertRepo
│   ├── service/
│   │   ├── __init__.py          # Service package exports
│   │   ├── monitoring.py        # MonitoringService & ThresholdRule
│   │   ├── alert.py             # AlertService lifecycle management
│   │   ├── notification.py      # NotificationService interface & implementations
│   │   └── reporting.py         # ReportingService metric analytics & reports
│   └── main.py                  # End-to-end simulation entry point
├── tests/
│   ├── __init__.py
│   ├── test_sensors.py          # Sensor hierarchy and unit conversion tests
│   ├── test_monitoring.py       # Threshold evaluation, alerts, & reporting tests
│   └── test_repositories.py     # Generic CRUD & specialized query tests
├── requirements.txt
└── README.md
```

4. Installation & Setup

Prerequisites

- Python 3.9 or higher (tested on Python 3.10+)

- `pip` package manager

1. Clone or Navigate to the Project

```
cd project_1_40213312
```

2. (Optional) Create and Activate a Virtual Environment

```
# Create virtual environment
python -m venv venv

# Activate on Windows (PowerShell):
.\venv\Scripts\Activate.ps1

# Activate on Linux/macOS:
source venv/bin/activate
```

3. Install Dependencies

```
pip install -r smart_building/requirements.txt
```

5. How to Run the Application

Execute the end-to-end demonstration script:

```
python smart_building/src/main.py
```

Demonstration Scenario Highlights

When `main.py` is executed, it automatically performs the following actions:

1. **Physical Facility Setup**: Instantiates `Building` ("Innovation Tower"), adds `Floor 2`, and provisions rooms (`Server Room`, `Executive Boardroom`).
2. **Sensor Deployment**: Instantiates and attaches 5 specialized sensors (`TemperatureSensor`, `HumiditySensor`, `CO2Sensor`, `MotionSensor`, `EnergySensor`) to rooms and registers them in the

repository.
3. **Nominal Ingestion**: Captures and processes normal operational telemetry without triggering alerts.
4. **Threshold Anomaly Simulation**:
   - Ingests **38.5°C** reading from the Server Room -> Triggers **CRITICAL Alert**.
   - Ingests **1250 ppm** CO_2 reading from the Boardroom -> Triggers **WARNING Alert**.
5. **Real-time Notifications**: `ConsoleNotificationService` logs formatted alerts to stdout.
6. **Incident Remediation**: Demonstrates the workflow of acknowledging and resolving active alerts.
7. **Diagnostic Facility Reports**: `ReportingService` computes sensor telemetry statistics (min, max, average) and full building health assessments.

6. How to Run Tests

The test suite covers unit tests for all domain models, repositories, and services using `pytest`:

```
pytest
```

For verbose output with individual test execution details:

```
pytest -v
```

7. Main Domain Entities & Components

Entity / Class	Layer	Description
`Building`	Domain	Represents an entire physical building; composites multiple `Floor` objects.
`Floor`	Domain	Represents a level within a building; composites multiple `Room` objects.
`Room`	Domain	Physical container that aggregates multiple `Sensor` instances.

Entity / Class	Layer	Description
`Sensor` (ABC)	Domain	Abstract base class managing sensor identity, status, heartbeat, and reading production.
`TemperatureSensor`	Domain	Specialized sensor for ambient temperature monitoring in Celsius (`C`).
`HumiditySensor`	Domain	Specialized sensor for relative humidity monitoring in percentage (`%`).
`CO2Sensor`	Domain	Specialized sensor for Carbon Dioxide concentration monitoring in ppm (`ppm`).
`MotionSensor`	Domain	Specialized sensor for occupancy/motion detection (`motion`).
`EnergySensor`	Domain	Specialized sensor for power consumption in kilowatt-hours (`kWh`).
`SensorReading`	Domain	Immutable record of a measurement with validation and unit conversion methods.
`Alert`	Domain	Models operational violations and tracks lifecycle states (`OPEN` , `ACKNOWLEDGED` , `RESOLVED`).
`Repository[T]` (ABC)	Repository	Generic CRUD abstract interface defining standard data access contracts.
`InMemoryRepository[T]`	Repository	Generic dictionary-backed repository implementation.
`SensorRepository`	Repository	Specialized repository with queries by room, status, and health.
`ReadingRepository`	Repository	Specialized repository with queries by sensor and timestamp filtering.
`AlertRepository`	Repository	Specialized repository with queries by sensor, severity, and status.
`NotificationService` (ABC)	Service	Abstract interface for dispatching notifications to stakeholders.

Entity / Class	Layer	Description
`ConsoleNotificationService`	Service	Concrete notification delivery printing formatted messages to standard output.
`EmailNotificationService`	Service	Concrete notification delivery simulating SMTP email dispatching.
`MonitoringService`	Service	Stream processing engine evaluating telemetry against threshold rules and detecting offline devices.
`AlertService`	Service	Manages creation, acknowledgment, and resolution of alerts with automated dispatch.
`ReportingService`	Service	Generates mathematical aggregations and comprehensive diagnostic health reports.

8. UML Class Diagram - Designed by Maral Torabi (Student ID: 40213312)

